



**Agentic Platform**

PLATAFORMA AGÉNTICA MULTI-TENANT

MANUAL 14 · MANUAL DE USUARIO

# Tutorial: proyecto CodeIgniter 4 con GitHub, agentes y MCP

Dirigido a: **Tenant admin / project owner que arranca su primer proyecto real**

Este tutorial monta, de cero y de punta a punta, un proyecto **CodeIgniter 4** respaldado por un repositorio de **GitHub**, con el **equipo de agentes CI4** que la plataforma trae de serie, y conectado por MCP a **Context7** (documentación de frameworks al día — los agentes dejan de alucinar APIs de versiones viejas) y a **Atlassian** (publicar documentación en Confluence y sincronizar issues de Jira). Al final tendrás un plan ejecutándose, el trabajo llegando como Pull Request a tu GitHub y la documentación fluyendo a Confluence.

El flujo completo es: **requisitos** → **proyecto** → **equipo** → **MCPs** → **tools a agentes** → **skills** → **primer plan** → **seguimiento** → **PR**. Cada paso indica la pantalla exacta, los valores exactos y el porqué. Donde hay una trampa conocida (egress, composer, tareas mal definidas) se avisa en el propio paso — este tutorial incorpora todo lo aprendido en las pruebas e2e reales de la plataforma.

Convenciones: sustituye tuorg/turepo por tu organización y repositorio de GitHub, y tuempresa.atlassian.net por tu instancia de Atlassian. Los secretos van SIEMPRE en Vault — nunca en formularios de configuración ni en ficheros del repo.

Generado · 2026-07-19

Idioma · Español

Versión · Producción

Confidencial · Comité de Dirección

# Contenido

---

- |    |                                                         |
|----|---------------------------------------------------------|
| 01 | Requisitos previos (una sola vez)                       |
| 02 | Crear el proyecto Codelgniter 4                         |
| 03 | Adoptar el equipo Codelgniter 4 built-in                |
| 04 | Revisar el Hub de Capacidad de los agentes              |
| 05 | MCP 1/2 — Context7 (documentación de frameworks al día) |
| 06 | MCP 2/2 — Atlassian (Confluence + Jira)                 |
| 07 | Otros MCP recomendados (y cuáles evitar)                |
| 08 | Asignar las tools MCP a los agentes correctos           |
| 09 | Skills: convertir el uso de los MCP en hábito           |
| 10 | El primer plan: qué pedir y cómo                        |
| 11 | Aprobar y ejecutar el plan                              |
| 12 | Seguimiento en vivo: La Oficina                         |
| 13 | El detalle de un run y su Replay                        |
| 14 | «Esperan tu decisión» y la validación humana del plan   |
| 15 | El Pull Request en tu GitHub                            |
| 16 | Troubleshooting específico de CI4 + MCP                 |

## 1 Requisitos previos (una sola vez)

Antes de crear el proyecto, deja listos estos cuatro requisitos:

- **Un proveedor LLM activo.** En **Plataforma** → **Proveedores LLM** debe existir al menos un proveedor con el candado en verde (usa el botón **Probar**). Cualquiera de los cuatro caminos vale (Claude SDK, Copilot, Azure Foundry u Ollama local/cloud): los agentes, el asistente y el córtex pueden usar cualquier proveedor del catálogo en cualquier momento, y el *vigía de credenciales* te avisará al inbox si alguno caduca.
- **Un PAT de GitHub** (Settings → Developer settings → Personal access tokens) con permisos **repo** (contenido y pull requests) sobre **tuorg/turepo**. Lo guardarás en Vault en el paso del proyecto — el sistema lo usa para hacer push de las ramas de plan y abrir los PR automáticos.
- **Un API token de Atlassian** (id.atlassian.com → Security → API tokens) de una cuenta con permiso de escritura en el espacio de Confluence y el proyecto de Jira que quieras usar.
- **El repositorio GitHub creado** con la rama **main** inicializada (un README basta). El sistema clona/empuja contra ese remoto; la rama base debe existir para que el PR automático tenga contra qué abrirse.

Con esto en su sitio, el resto del tutorial es todo dentro del panel.

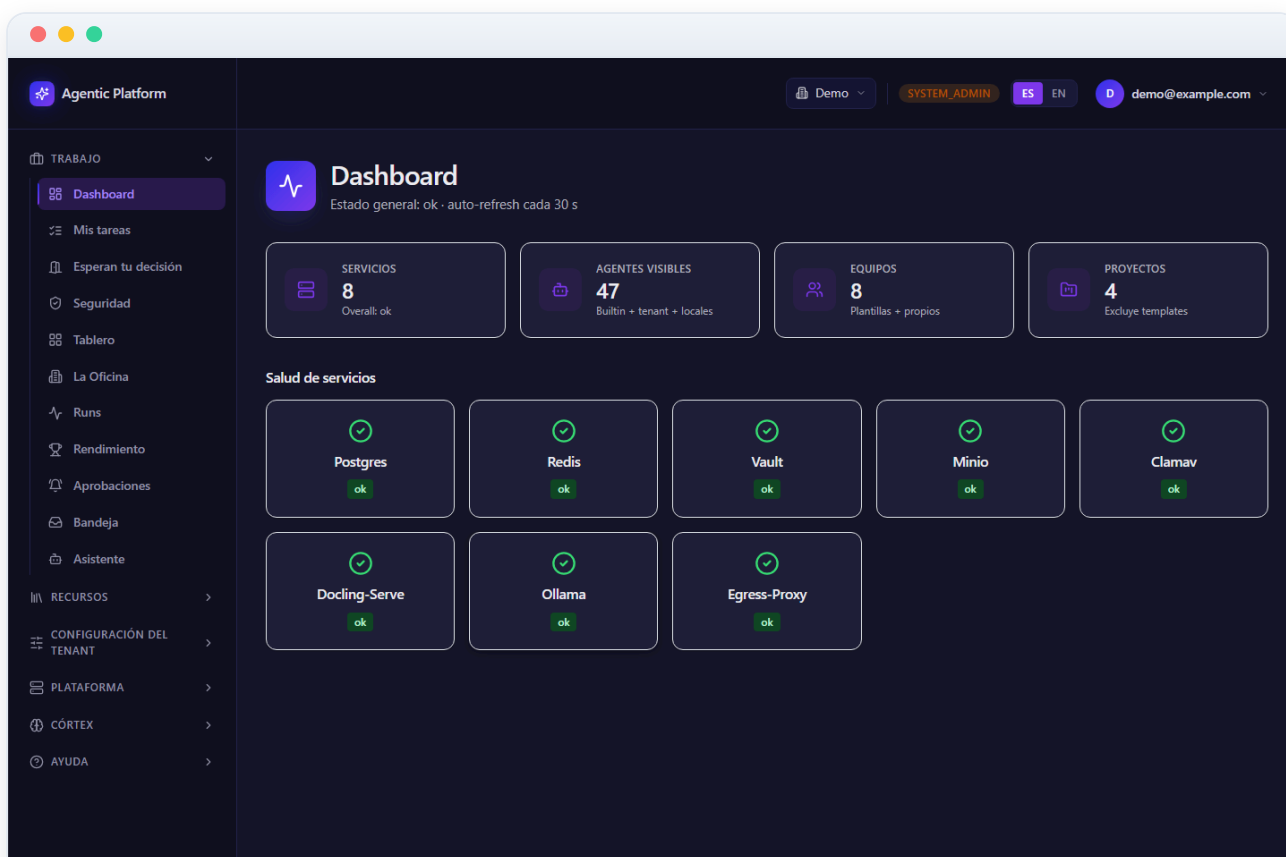


Figura 14.1 — Requisitos previos (una sola vez)

## 2 Crear el proyecto Codelgniter 4

En **Recursos** → **Proyectos**, pulsa «Nuevo proyecto» y rellena:

- **Nombre:** p. ej. `Mi API CI4` . El slug se deriva solo.
- **Stack:** `php` — determina el runtime template por defecto y qué equipo built-in encaja.
- **Runtime template:** `php-phpunit` . Es el contenedor donde los agentes ejecutan el toolchain del proyecto (composer, phpunit, spark) vía `stack_exec` — los workers nunca ejecutan código de usuario (Principio 2), lo hace este runtime efímero con red restringida.
- **Git remoto:** `https://github.com/tuorg/turepo.git` , rama por defecto `main` . La credencial (el PAT) se referencia por Vault: guarda el token en la ruta que el formulario te indique (convención `vault:secret/data/git/<proyecto>` ) — el formulario nunca almacena el token.
- **Comandos permitidos** (allowlist deny-by-default de `shell_exec` / `stack_exec` ): `composer` , `php` , `vendor/bin/phpunit` . Sin esta lista los agentes no pueden ni instalar dependencias ni correr tests: añádela ahora.
- **Dominios permitidos:** déjalo vacío de momento; añadiremos `mcp.context7.com` en el paso de Context7.

Guarda. Cada plan de este proyecto se materializará como una rama `plan/<id>-<slug>` en tu GitHub, con commits que llevan los trailers `Plan-Id` / `Task-Id` / `Execution-Id` , y un PR automático al completarse (Principio 5).

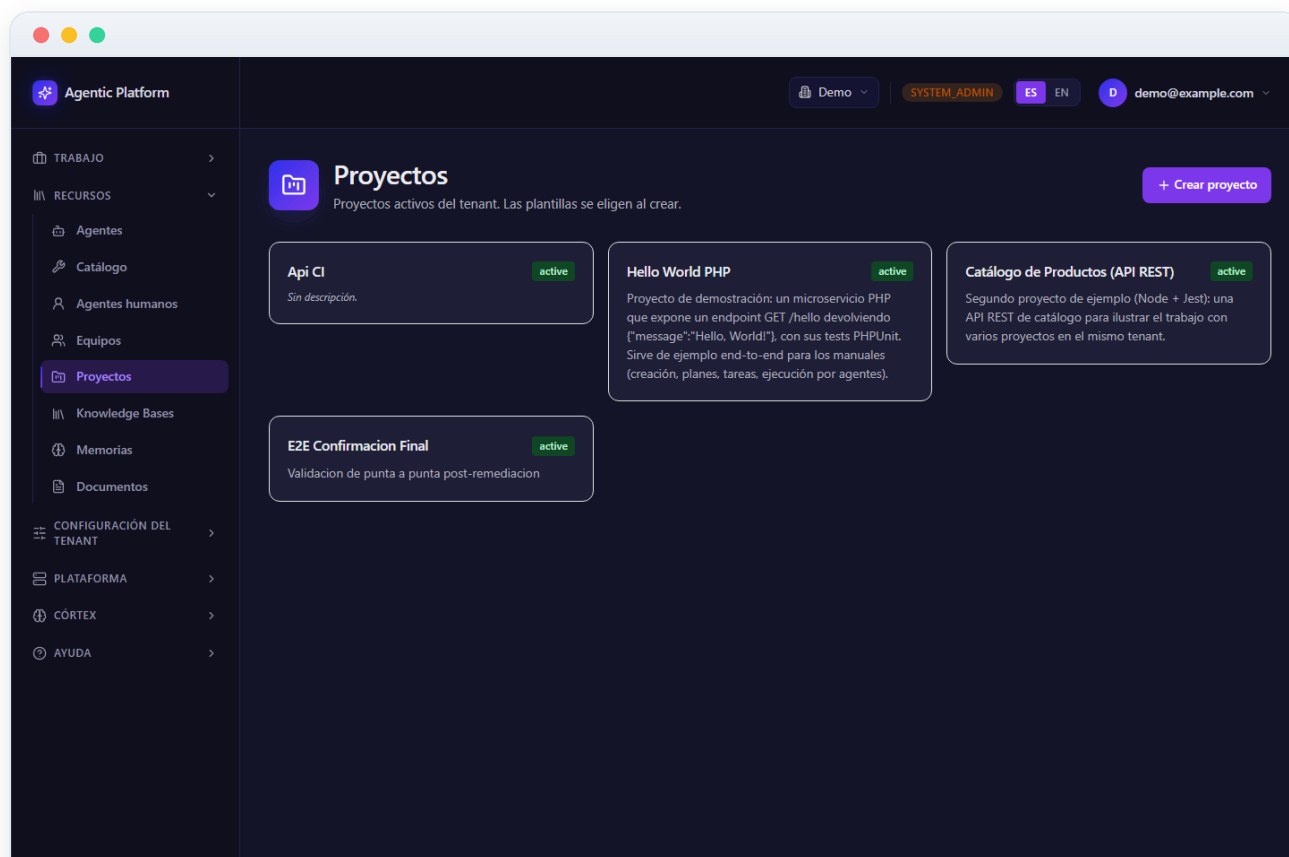


Figura 14.2 — Crear el proyecto Codelgniter 4

### 3 Adoptar el equipo Codelgniter 4 built-in

La plataforma trae un **equipo CI4 de 10 agentes** mantenido de serie (PM líder, arquitecto, backend devs, frontend, QA, reviewer, DevOps, security y technical writer) con personas y prompts especializados en Codelgniter 4. En **Recursos** → **Equipos**, localiza «**Codelgniter 4**» en la pestaña de built-in y pulsa «**Adoptar**» (ADR 0066): la adopción crea TU copia del equipo — puedes ajustar personas, modelos o skills sin tocar el original, y el original puede evolucionar sin pisarte.

En el diálogo de adopción, asócialo a tu proyecto **Mi API CI4**. Desde ese momento la **asignación por rol** (ADR 0091) despachará cada tarea del plan al agente del rol correspondiente: las tareas **backend\_dev** al Backend Dev del equipo, las **qa** al QA, etc.

**Modelo LLM:** los agentes heredan el modelo por la cadena plataforma → proyecto → agente (ADR 0055/0082). Si quieres que este proyecto use un modelo concreto (p. ej. el que mejor salga en tu pantalla **Rendimiento**), fíjalo a nivel de proyecto y deja que los agentes hereden; sobrescribe por agente solo cuando tengas un motivo (p. ej. un reviewer con un modelo más potente).

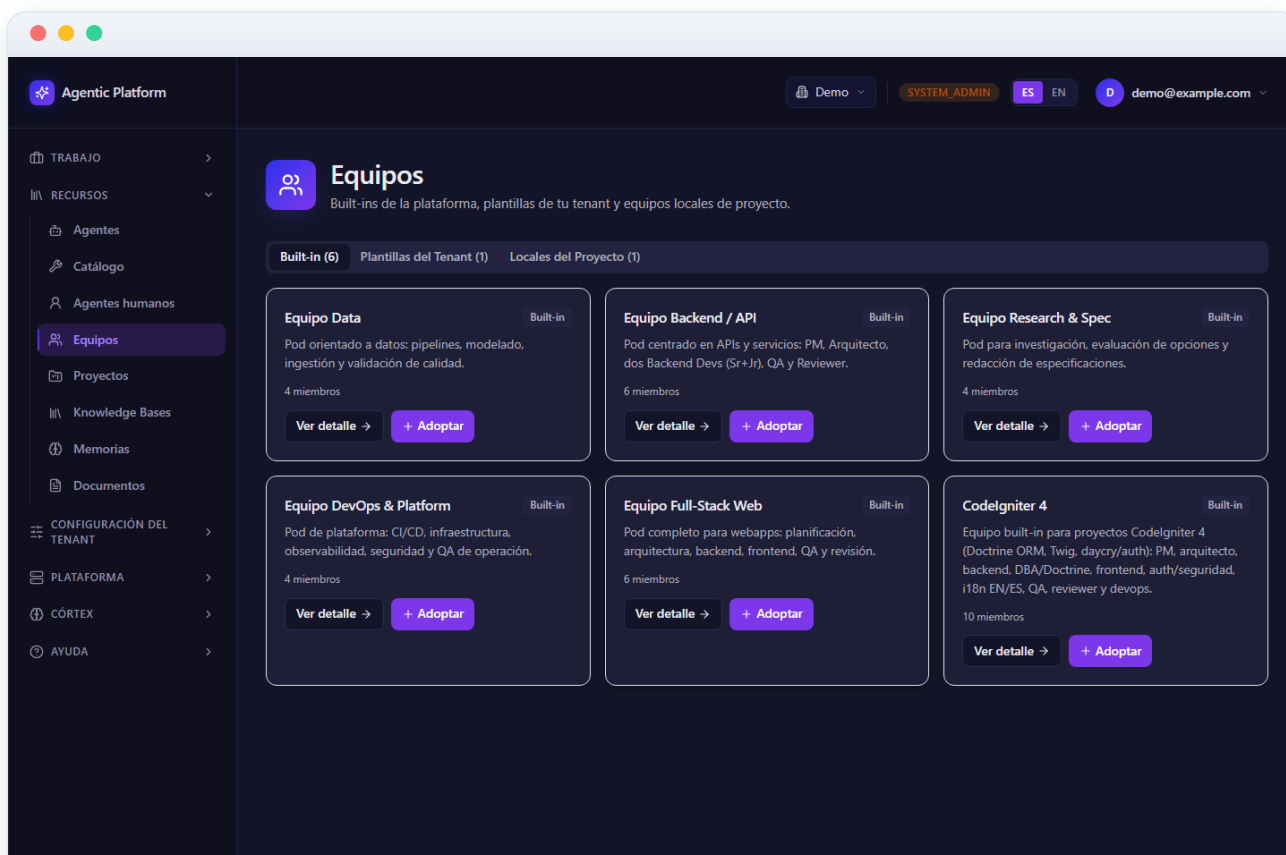


Figura 14.3 — Adoptar el equipo Codelgniter 4 built-in

## 4 Revisar el Hub de Capacidad de los agentes

Abre la ficha de un agente del equipo (p. ej. el **Backend Dev**). Su **Hub de Capacidad** resume las cuatro vías con su estado real:

- **SER** — su persona CI4 (system prompt bilingüe, modelo heredado, temperatura).
- **SABER** — las knowledge bases de rol/stack asignadas (el corpus PHP/CI4 curado).
- **HACER** — sus tools: verás las builtin (ficheros, tests, red) y aquí añadiremos las MCP en unos pasos.
- **RECORDAR** — el scope de memoria (para un equipo, `team_shared` es lo razonable: lo aprendido lo comparte el equipo).

No cambies nada aún — este paso es para que reconozcas la pantalla: volveremos aquí a asignar las tools MCP y una skill.

Regla de oro del Hub: *lo que no está asignado aquí, el agente no lo ve en sus runs* — la allowlist efectiva de un run es la intersección de lo asignado al agente con el modo de ejecución.

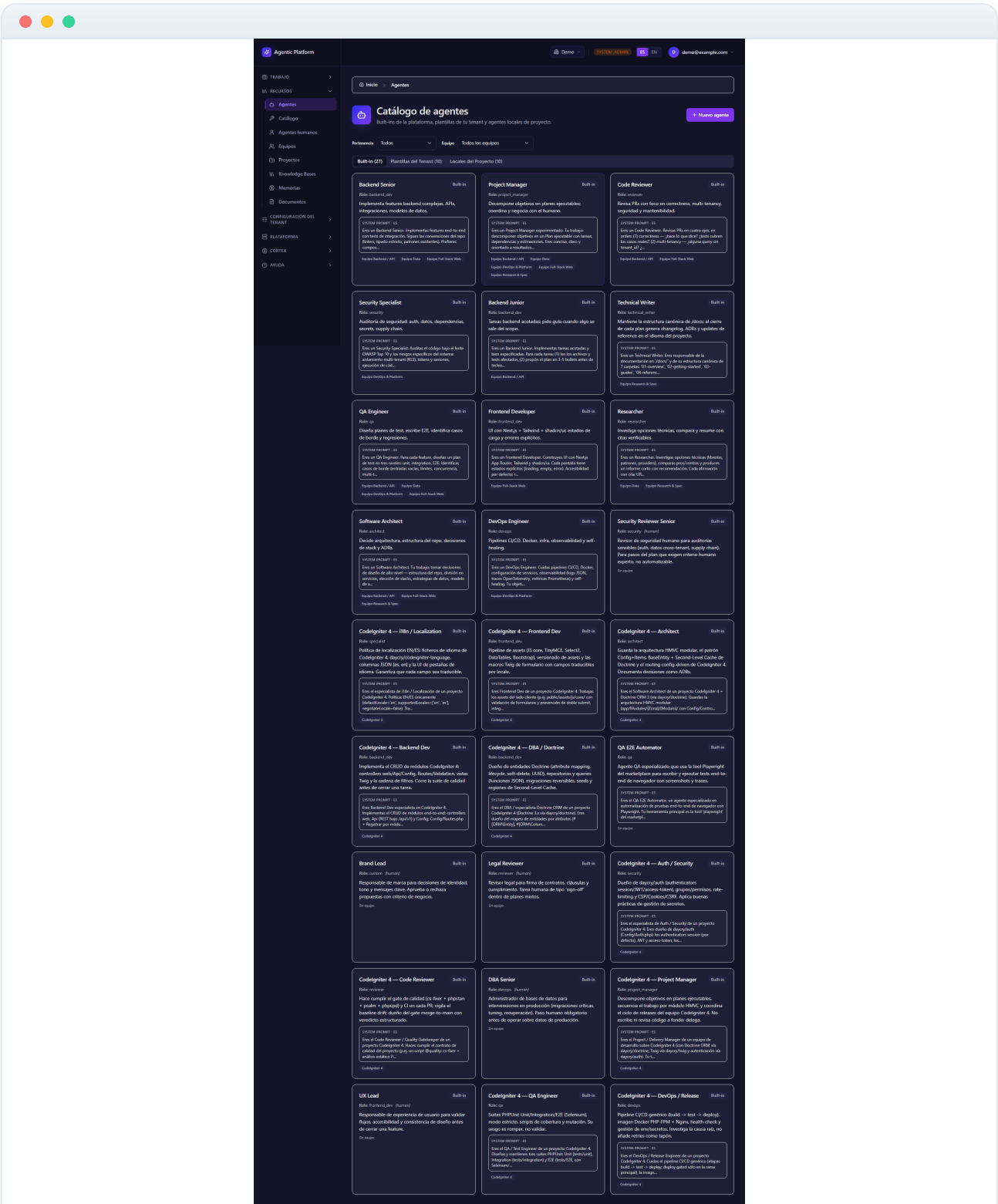


Figura 14.4 — Revisar el Hub de Capacidad de los agentes

## 5 MCP 1/2 — Context7 (documentación de frameworks al día)

Context7 es un MCP remoto público que sirve documentación ACTUALIZADA de frameworks — para un proyecto CI4 significa que los agentes consultan la API real de Codelgniter 4.x en vez de recordarla (adiós a los métodos alucinados de versiones viejas). Entra en tu proyecto → pestaña **MCP servers** → «Añadir MCP server»:

- **Nombre:** `context7`
- **Transporte:** `streamable_http`
- **URL:** `https://mcp.context7.com/mcp`
- **Credencial:** opcional (funciona sin cuenta con rate limits). Con API key, guárdala en Vault como `CONTEXT7_API_KEY` en `vault:secret/data/mcp/context7/<project_id>` — viajará como cabecera.

⚠ **Egress obligatorio:** los runs de agentes salen a internet por el proxy allowlisted (deny-by-default). Vuelve a la configuración del proyecto y añade `mcp.context7.com` a los **dominios permitidos**; sin esto el transporte devolverá `403 Filtered` y el server nunca conectará (verás el motivo exacto en el step `mcp_wire` del visor de runs).

Pulsa «Probar» — debe listar `resolve-library-id` y `get-library-docs` — y después «Importar tools»: quedan en el catálogo como `context7.resolve-library-id` y `context7.get-library-docs`.

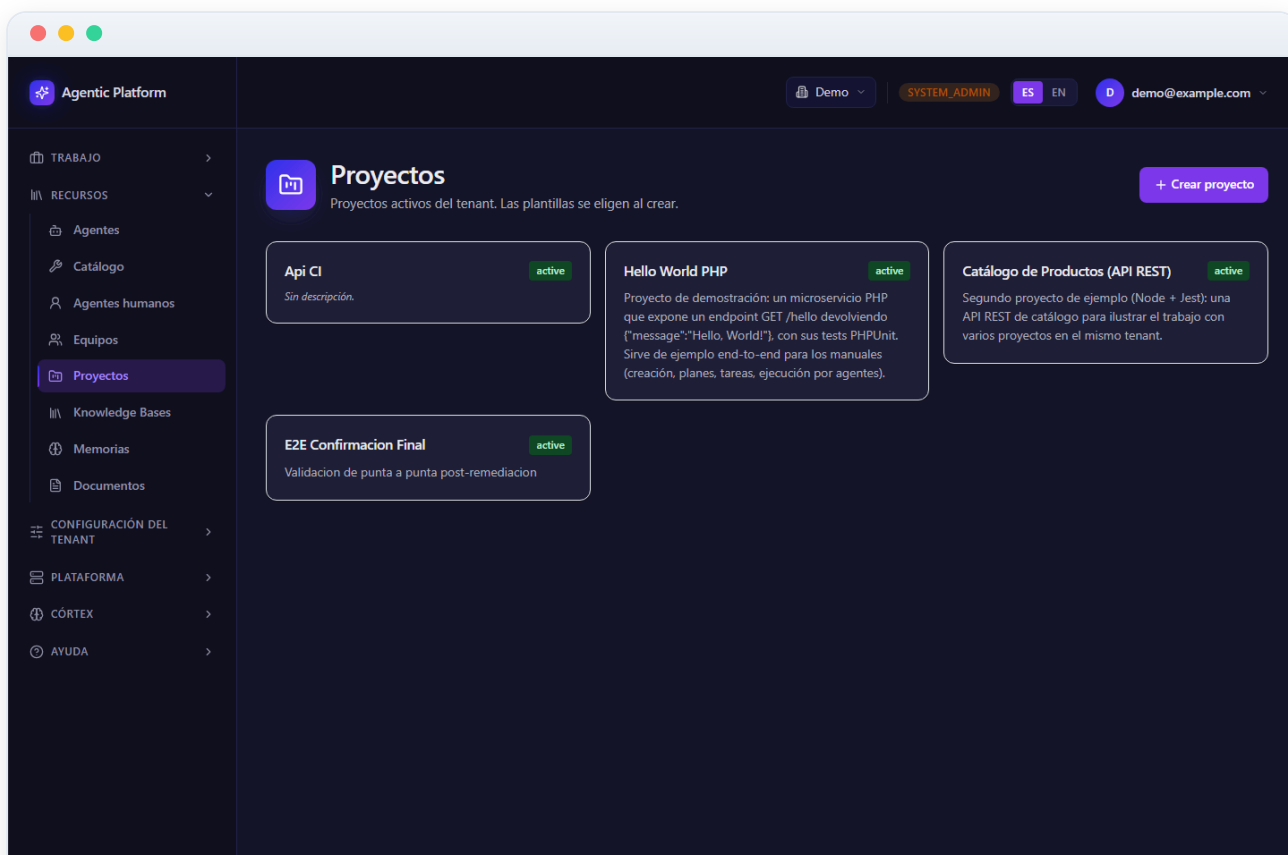


Figura 14.5 — MCP 1/2 — Context7 (documentación de frameworks al día)



## 6 MCP 2/2 — Atlassian (Confluence + Jira)

Para agentes autónomos usa el server MCP **self-hosted** de Atlassian con API token (el remoto oficial exige OAuth interactivo de un humano, que no encaja en runs). Despliega el sidecar junto al stack, conectado a la red de agentes:

```
docker-compose.override.yml :
```

services:

```
mcp-atlassian:
  image: ghcr.io/sooperset/mcp-atlassian:latest
  environment:
    JIRA_URL: https://tuempresa.atlassian.net
    CONFLUENCE_URL: https://tuempresa.atlassian.net/wiki
    JIRA_USERNAME: bot@tuempresa.com
    JIRA_API_TOKEN: ${ATLASSIAN_API_TOKEN}
    CONFLUENCE_USERNAME: bot@tuempresa.com
    CONFLUENCE_API_TOKEN: ${ATLASSIAN_API_TOKEN}
  command: ["--transport", "streamable-http", "--port", "9000"]
  networks: [agentic-agents]
```

Levántalo ( `docker compose up -d mcp-atlassian` ) y declara el server en el proyecto:

- **Nombre:** `atlassian` · **Transporte:** `streamable_http` · **URL:** `http://mcp-atlassian:9000/mcp`

Al ser un hostname INTERNO de la red de agentes, no pasa por el proxy de egress (la declaración del server en el proyecto es la autorización). «**Probar**» debe listar las tools de Confluence y Jira; «**Importar tools**» las materializa como `atlassian.confluence_create_page` , `atlassian.jira_transition_issue` , `atlassian.jira_search` ... Este flujo exacto está validado e2e en esta plataforma: un run real publicó una página en Confluence y transicionó una issue de Jira encadenando la URL devuelta.

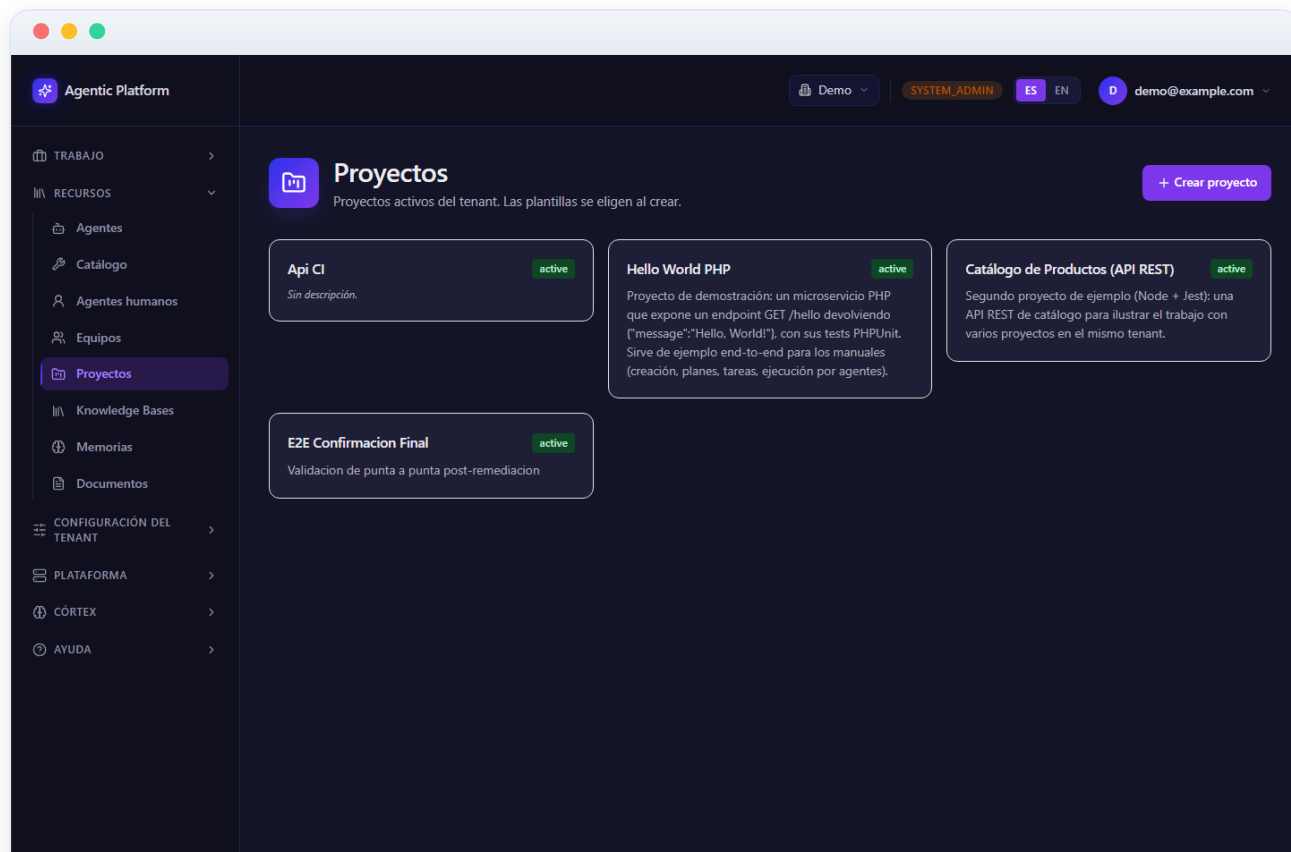


Figura 14.6 — MCP 2/2 — Atlassian (Confluence + Jira)

## 7 Otros MCP recomendados (y cuáles evitar)

Con el mismo flujo (declarar → probar → importar → asignar) puedes añadir más servers. Recomendaciones honestas para un proyecto CI4:

- **GitHub** (plantilla del catálogo): tools de issues y PRs — útil si gestionas el backlog en GitHub Issues además del PR automático que ya tienes. Guarda el `GITHUB_TOKEN` en la ruta de Vault que la plantilla te indique.
- **Un MCP propio** de tu empresa (ERP, CRM, base de conocimiento interna): con FastMCP son ~30 líneas de Python; desplégalo como el sidecar de Atlassian (red `agentic-agents`, transporte `streamable-http`) y sigue el flujo universal. La receta completa está en `docs/03-guides/recetas-mcp-tools-skills.md`.
- **Docling** (interno): la conversión de documentos ya está integrada en la plataforma vía la tool `document_convert` — no necesitas declararlo por MCP.

⚠ **Evita por ahora** las plantillas históricas con transporte `stdio` que referencian binarios ( `npx ...`, `uvx ...` ): esos ejecutables no están empaquetados en la imagen del runtime y el server no arrancará (decisión pendiente en el ADR 0117). Regla práctica: **prefiere siempre servers HTTP** ( `streamable-http` / `sse` ) — remotos o sidecars — que son los validados e2e.

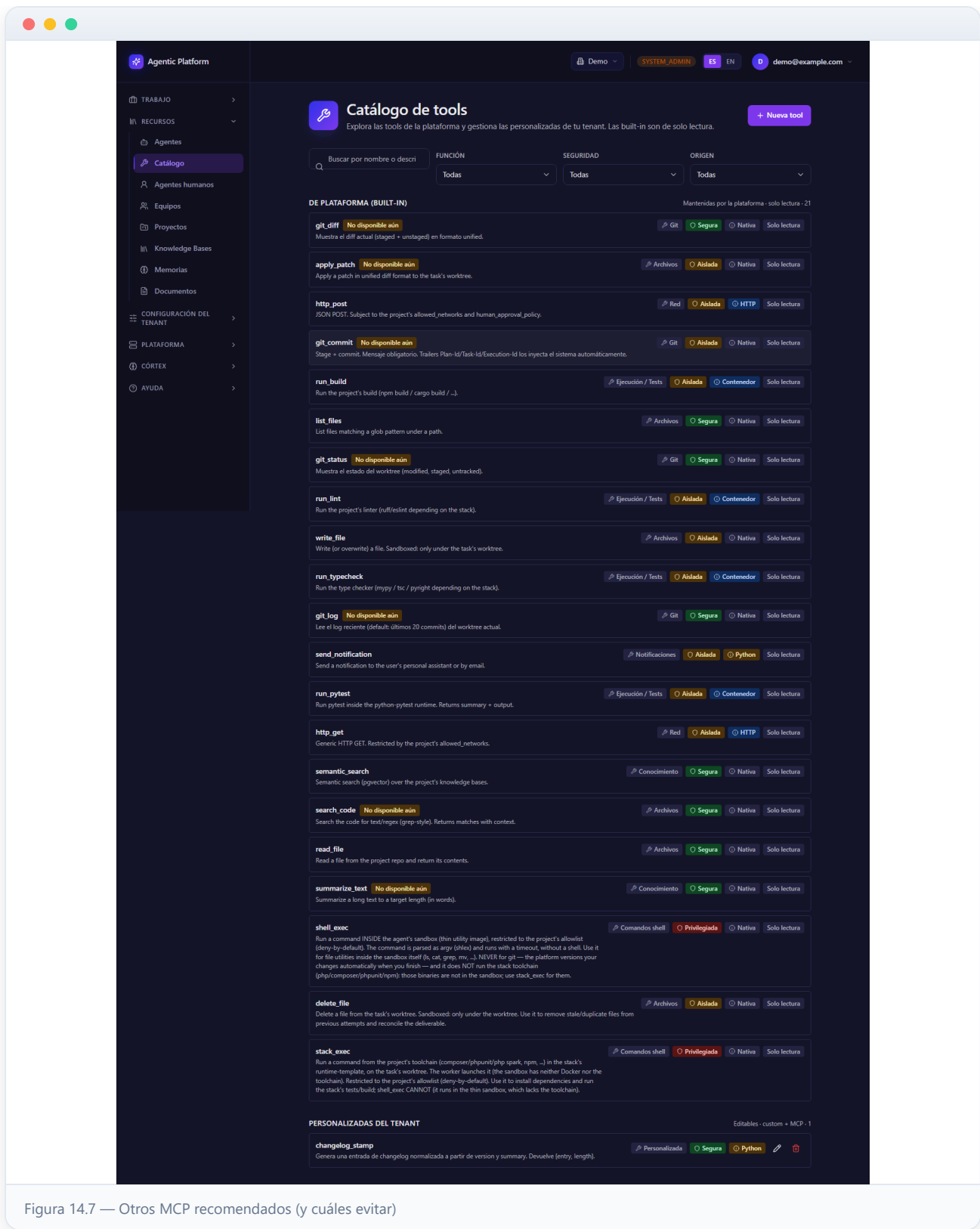


Figura 14.7 — Otros MCP recomendados (y cuáles evitar)

## 8 Asignar las tools MCP a los agentes correctos

Importar las tools NO basta: cada agente solo ve las que tiene asignadas (la allowlist filtra el resto). Vuelve a la ficha de cada agente → sección **Tools** → pestaña **Avanzadas** y asigna con criterio de mínimo privilegio:

- **Backend Dev / Frontend Dev / Arquitecto:** `context7.resolve-library-id` + `context7.get-library-docs` — son quienes escriben código y consultan APIs.
- **Technical Writer:** `atlassian.confluence_create_page` (+ `context7.*` si documenta APIs) — es quien publica documentación.
- **Project Manager:** `atlassian.jira_search` + `atlassian.jira_transition_issue` — es quien sincroniza el estado con Jira.

No asignes todas las tools a todos: menos superficie es más seguridad Y mejor foco del modelo (menos tools anunciadas = elecciones más acertadas). Si un run intenta llamar una tool no asignada verás `unknown tool` en su timeline — la señal de que falta esta capa.

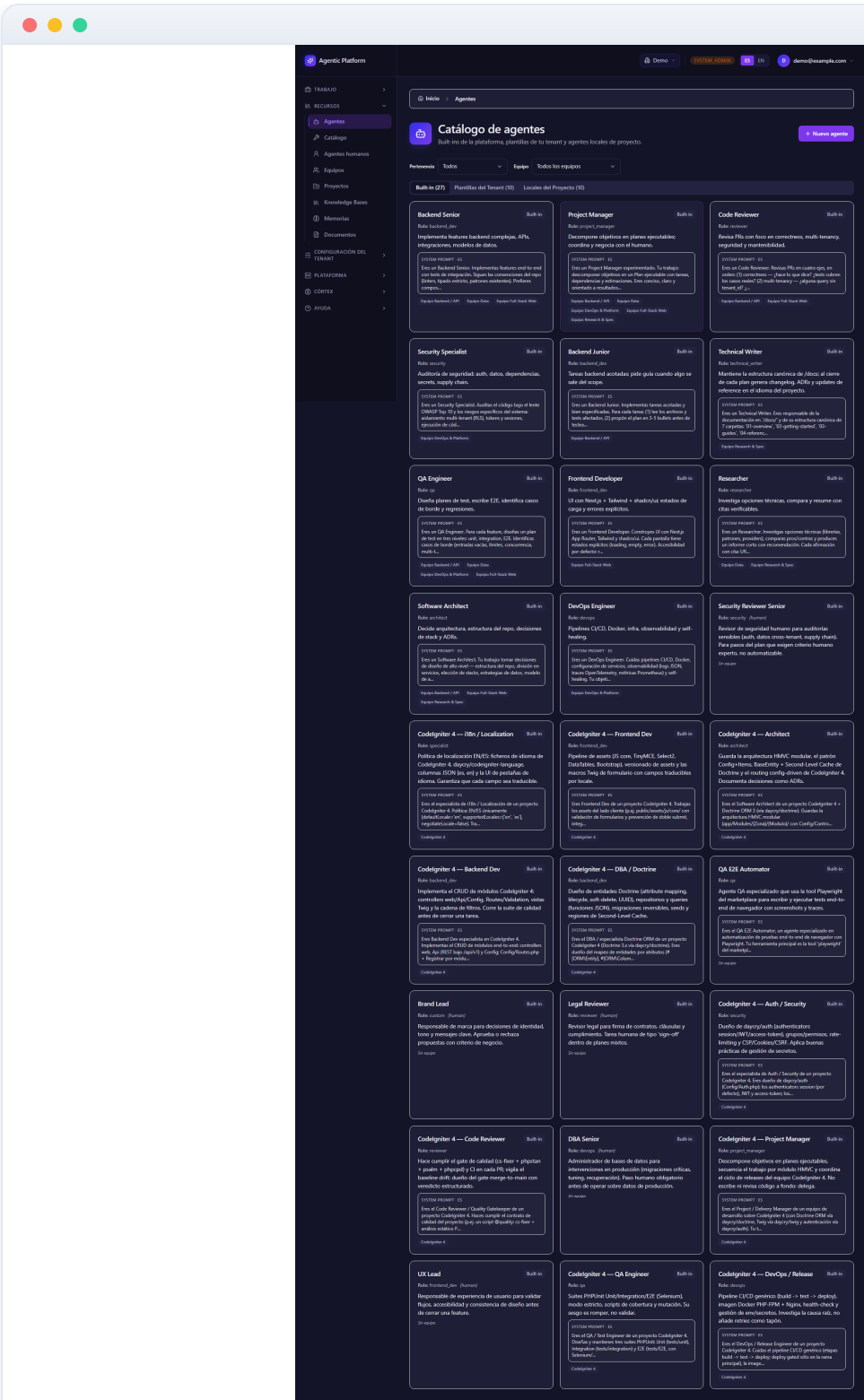


Figura 14.8 — Asignar las tools MCP a los agentes correctos

## 9 Skills: convertir el uso de los MCP en hábito

Las tools dan la CAPACIDAD; las **skills** (fragmentos de system prompt asignables, ADR 0050) fijan el HÁBITO de usarlas sin repetirlo en cada tarea. Crea estas dos vía `POST /api/skills` (rol tenant admin) y asígnalas en la sección **Skills** de la ficha de cada agente:

**Skill «Docs al día con Context7»** (para los devs y el arquitecto):

Antes de usar una API de CodeIgniter 4 (o de cualquier librería) de la que no estés 100% seguro, resuelve la librería con `context7.resolve-library-id` y consulta `context7.get-library-docs` con el topic concreto. Prefiere SIEMPRE la firma devuelta por la documentación a tu memoria.

**Skill «Sincronizar Jira al cerrar»** (para el PM):

Cuando una tarea con issue de Jira asociada quede hecha, transiciónala al estado correspondiente con `atlassian.jira_transition_issue` e incluye en el comentario el enlace al resultado (PR, página de Confluence).

Recuerda: una skill no otorga tools — el agente debe tener la tool asignada ADEMÁS de la skill (el campo `required_tools` documenta la dependencia). Este mecanismo está validado e2e: en las pruebas reales, un agente usó una custom tool que su tarea nunca mencionaba, inducido solo por el fragment de su skill.

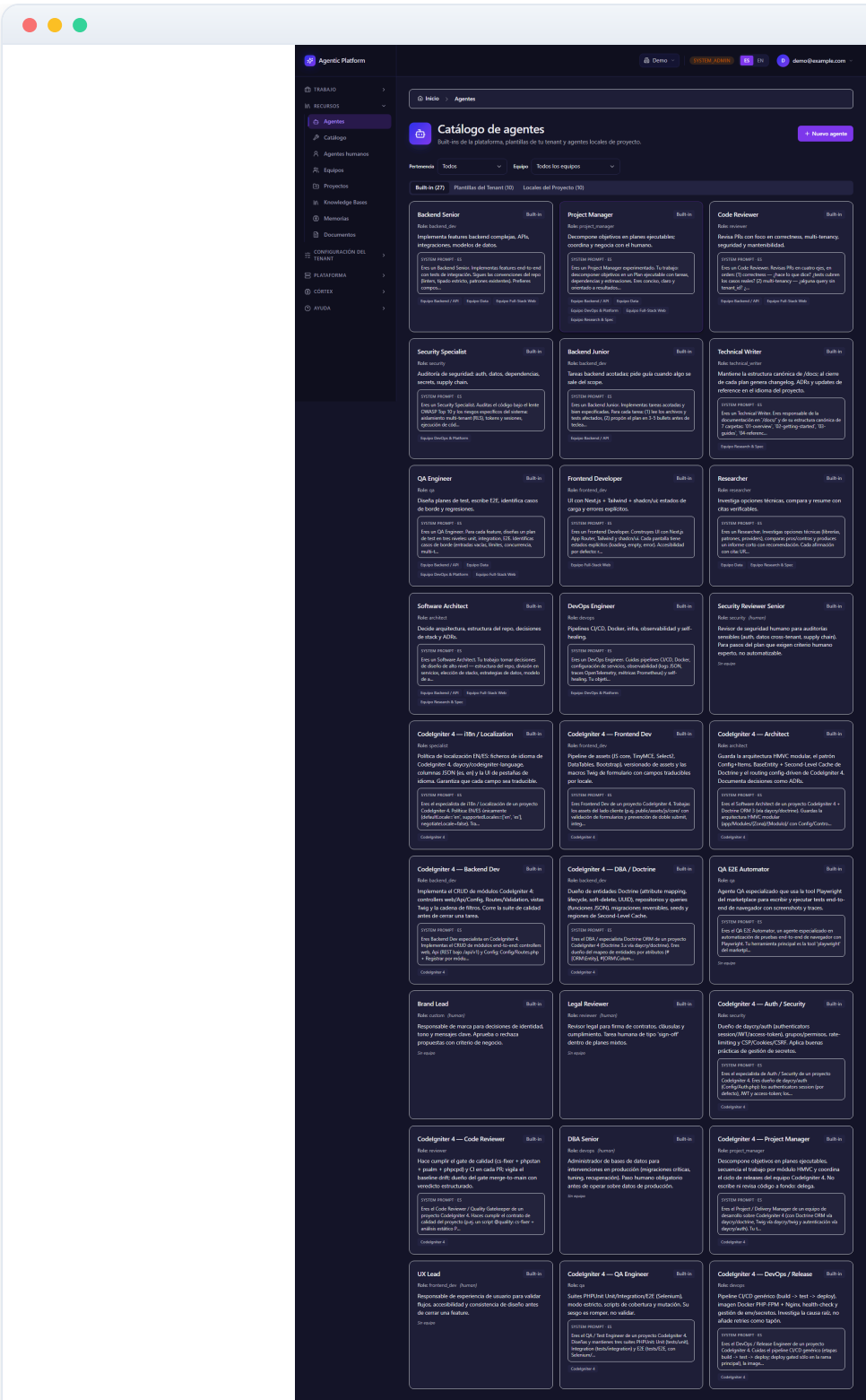


Figura 14.9 — Skills: convertir el uso de los MCP en hábito

## 10 El primer plan: qué pedir y cómo

Con todo cableado, crea el primer plan. Dos caminos:

- **Chat de planning** (recomendado): desde el proyecto, abre el chat de planificación y describe el objetivo en lenguaje natural — p. ej. «Inicializa un proyecto Codelgniter 4 con composer, configura el entorno base (.env, rutas, un controlador Health con test), documenta la instalación en Confluence (espacio DOCS) y deja la issue CI4-1 de Jira en Done». El planner genera las tareas con roles, dependencias y **criterios de aceptación**, y las inserta en el Kanban asignadas por rol.
- **Especificación manual**: crea el plan con sus tareas a mano si prefieres control total.

**Reglas de oro para que los runs converjan** (aprendidas en las pruebas e2e):

- **Tareas autocontenidas**: si una tarea dice «publica el fichero X», X debe existir o la propia tarea debe crearlo antes — un insumo inexistente hace que el agente agote sus iteraciones buscándolo.
- **Nombra las tools MCP por su nombre exacto** con namespace ( `atlassian.confluence_create_page` ) cuando la acción sea puntual de esa tarea.
- **Criterios de aceptación verificables**: el reviewer y la self-review certifican contra ellos; «se invocó atlassian.X con space DOCS» es verificable (la self-review VE las tool calls del run), «que quede bonito» no.

The screenshot displays the Agentic Platform interface. On the left is a sidebar with navigation options: TRABAJO (Dashboard, Mis tareas, Esperar tu decisión, Seguridad, Tablero, La Oficina, Runs, Rendimiento, Aprobaciones, Bandeja, Asistente), RECURSOS, CONFIGURACIÓN DEL TENANT, PLATAFORMA, CORTEX, and AYUDA. The main area is titled 'Tablero' and shows a Kanban board with plans and tasks.

**Planes** (9 planes):

- Prueba MCP Atlassian v5 - final (E2E Confirmacion Final, pending\_human\_validation)
- Prueba MCP Atlassian v4 (E2E Confirmacion Final, cancelled)
- Prueba custom tool + skill (E2E Confirmacion Final, pending\_human\_validation)
- Prueba MCP Atlassian v2 (E2E Confirmacion Final, cancelled)
- Prueba MCP Atlassian (E2E Confirmacion Final, cancelled)
- Plan E2E de confirmacion (E2E Confirmacion Final, pending\_human\_validation)
- Página de estado — demo del ciclo de correcciones (Hello World PHP, rejected)
- MVP — API Hello World en PHP (Hello World PHP, pending\_human\_validation)
- API Codelgniter 4 — endpoints públicos v1 y comando de diagnóstico (Api CI, completed)

**Tareas — Prueba MCP Atlassian v5 - final** (Tiempo real, 1 tarea):

| Backlog    | Ready      | En curso   | Pendiente de aprobación | Revisión   | Bloqueada  | Hecho                                                                                |
|------------|------------|------------|-------------------------|------------|------------|--------------------------------------------------------------------------------------|
| 0          | 0          | 0          | 0                       | 0          | 0          | 1                                                                                    |
| Sin tareas | Sin tareas | Sin tareas | Sin tareas              | Sin tareas | Sin tareas | Crear la bienvenida, publicarla en Confluence y sincronizar Jira (medium) 1) Crea... |

Additional task: Cancelada (0, Sin tareas)

Figura 14.10 — El primer plan: qué pedir y cómo



## 11 Aprobar y ejecutar el plan

El plan nace en borrador/pendiente de aprobación. La transición a **aprobado** es un gate humano explícito (botón **Aprobar** — el PUT genérico la rechaza por diseño), y con la aprobación puedes **iniciar la ejecución**. A partir de ahí el ciclo es autónomo:

- El orchestrator despacha cada tarea **ready** al agente de su rol; el runtime abre el worktree del plan, conecta los MCP declarados (step **mcp\_wire** en el timeline del run) y el agente trabaja: percibe → recuerda → planifica → actúa (tools) → observa → reflexiona.
- Cada run pasa su **self-review** (que certifica contra los criterios de aceptación viendo el código real y las tool calls) y después el **AI reviewer** del equipo revisa el trabajo con su propio run.
- Los commits van llegando a la rama **plan/...** de tu GitHub con sus trailers.

Si algo exige decisión humana (una acción sensible con política de aprobación, un run que no converge), la tarea espera en los gates correspondientes — lo verás en el paso de seguimiento.

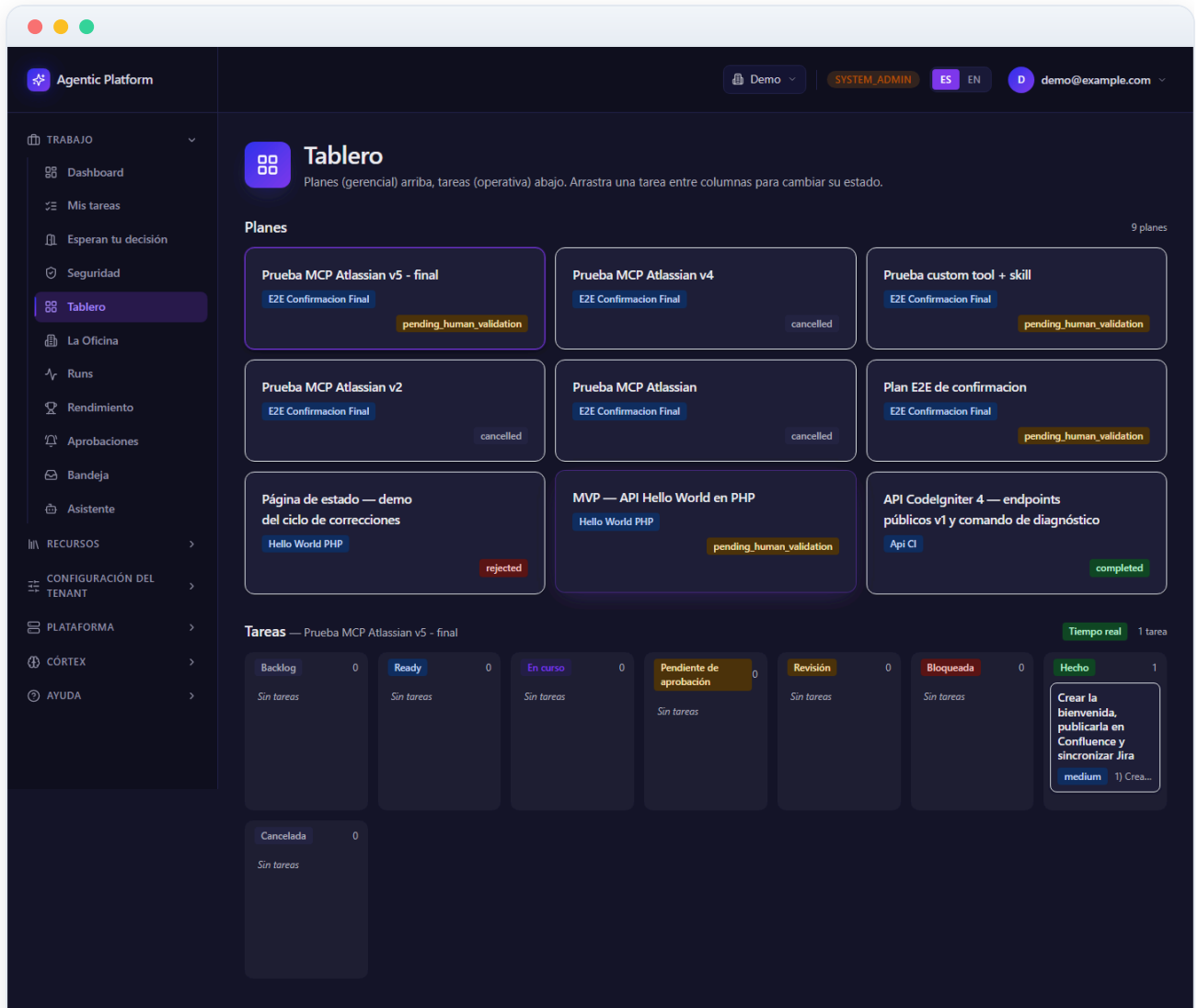


Figura 14.11 — Aprobar y ejecutar el plan

## 12 Seguimiento en vivo: La Oficina

**Trabajo** → **La Oficina** es el tenant en vivo como piso 2D donde TODO mapea a telemetría real: cada mesa es un plan con runs activos, cada personaje un agente con su estado verdadero (🖥️ trabajando, 🔍 revisando, 🔄 dando vueltas en un bucle detectado, 🪑 esperando a un humano, 😴 sin trabajo), y la burbuja muestra su tarea. Clic en un personaje → su run real.

Durante tu primer plan CI4 verás al Backend Dev sentarse en la mesa del plan, al Reviewer acercarse cuando le toque revisar, y — si configuraste bien los MCP — nada de personajes mareados: un agente dando vueltas suele significar una tarea mal definida (insumo inexistente) o una tool que no conecta.

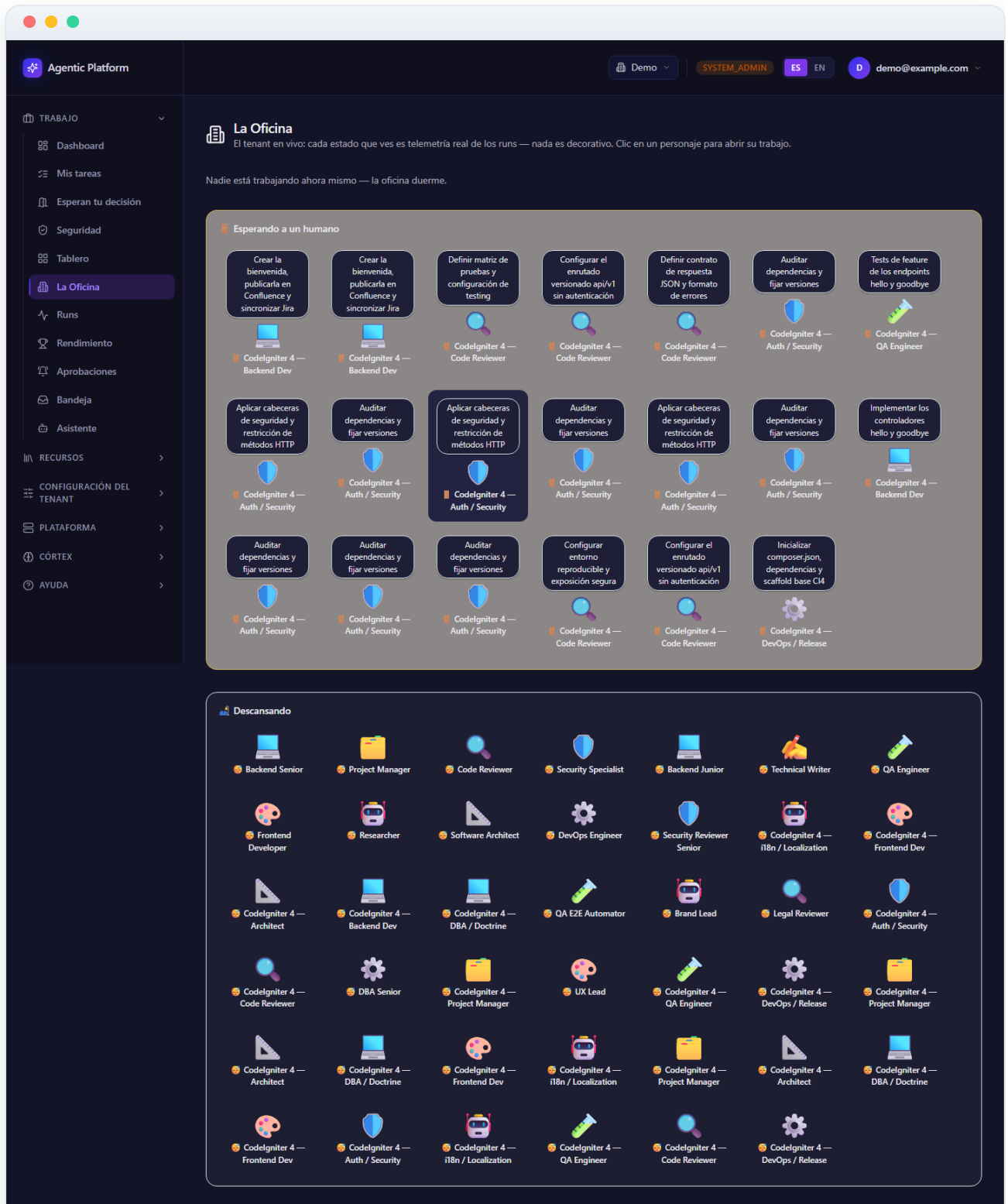


Figura 14.12 — Seguimiento en vivo: La Oficina

## 13 El detalle de un run y su Replay

En **Trabajo** → **Runs** tienes el historial completo; entra en cualquier ejecución para ver su **timeline**: cada paso del grafo (percepción, memoria, llamadas al modelo con tokens y coste, cada `tool_call` con argumentos y resultado, la conexión `mcp_wire` de cada server MCP, y el veredicto de la self-review). Es tu herramienta de diagnóstico número uno:

- `mcp_wire` en error → el server MCP no conectó (el motivo exacto está en el step: egress, URL, credencial).
- `unknown tool` → la tool no está asignada al agente (paso de asignación).
- Bucles de `read_file` / `list_files` → tarea no autocontenida.

El botón  **Replay** reproduce el run paso a paso (play/pausa/velocidad/scrubber) — perfecto para entender qué hizo el agente y para enseñar el sistema a tu equipo.

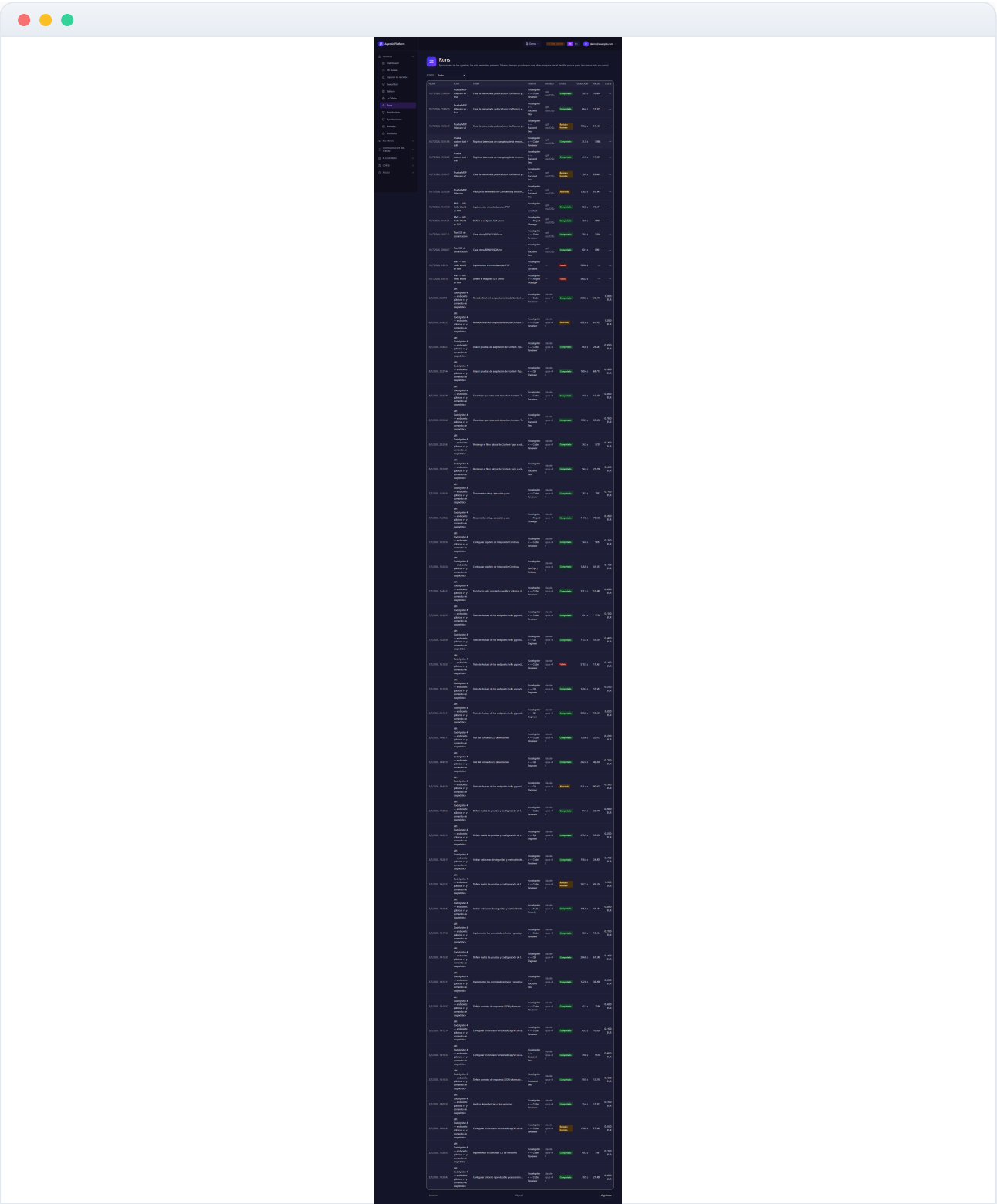


Figura 14.13 — El detalle de un run y su Replay

## «Esperan tu decisión» y la validación humana del plan

**Trabajo** → **Esperan tu decisión** reúne en una sola lista, por antigüedad, todo lo que está parado hasta que actúes: planes pendientes de validación humana, acciones sensibles por aprobar, y runs escalados o aparcados. Con más de 24 h de espera la edad se pinta en rojo. Cada fila te lleva a la pantalla real donde se resuelve.

Cuando todas las tareas del plan CI4 terminen, el plan pasa a **pendiente de validación humana** (los tests humanos son a nivel de PLAN — Principio 7): pruébalo (el controlador Health responde, los tests pasan, la página de Confluence existe, la issue de Jira está en Done) y valida. Con tu validación el plan se completa y...

**Esperan tu decisión**  
Todo lo que está parado hasta que un humano actúe, lo más antiguo primero.

| Tarea                                                            | Run escalado - Api CI | Plan por validar - Hello World PHP |
|------------------------------------------------------------------|-----------------------|------------------------------------|
| Inicializar composer.json, dependencias y scaffold base CI4      | 28 d                  |                                    |
| Inicializar composer.json, dependencias y scaffold base CI4      | 28 d                  |                                    |
| Configurar el enrutado versionado api/v1 sin autenticación       | 19 d                  |                                    |
| Configurar entorno reproducible y exposición segura              | 19 d                  |                                    |
| Auditar dependencias y fijar versiones                           | 19 d                  |                                    |
| Auditar dependencias y fijar versiones                           | 19 d                  |                                    |
| Auditar dependencias y fijar versiones                           | 19 d                  |                                    |
| Implementar los controladores hello y goodbye                    | 18 d                  |                                    |
| Auditar dependencias y fijar versiones                           | 18 d                  |                                    |
| Auditar dependencias y fijar versiones                           | 18 d                  |                                    |
| Aplicar cabeceras de seguridad y restricción de métodos HTTP     | 18 d                  |                                    |
| Aplicar cabeceras de seguridad y restricción de métodos HTTP     | 17 d                  |                                    |
| Auditar dependencias y fijar versiones                           | 17 d                  |                                    |
| Aplicar cabeceras de seguridad y restricción de métodos HTTP     | 16 d                  |                                    |
| Tests de feature de los endpoints hello y goodbye                | 16 d                  |                                    |
| Auditar dependencias y fijar versiones                           | 16 d                  |                                    |
| Definir contrato de respuesta JSON y formato de errores          | 16 d                  |                                    |
| Configurar el enrutado versionado api/v1 sin autenticación       | 16 d                  |                                    |
| Definir matriz de pruebas y configuración de testing             | 16 d                  |                                    |
| MVP — API Hello World en PHP                                     |                       | 1 d                                |
| Plan EZE de confirmacion                                         |                       | 1 d                                |
| Crear la bienvenida, publicarla en Confluence y sincronizar Jira |                       | 22 h                               |
| Prueba custom tool + skill                                       |                       | 22 h                               |
| Crear la bienvenida, publicarla en Confluence y sincronizar Jira |                       | 22 h                               |
| Prueba MCP Atlassian v5 - final                                  |                       | 22 h                               |

Figura 14.14 — «Esperan tu decisión» y la validación humana del plan

## 15 El Pull Request en tu GitHub

...al completarse, el sistema abre automáticamente el **Pull Request** en `tuorg/turepo` : rama `plan/<id>-<slug>` contra `main` , con el cuerpo autogenerado (resumen del plan y sus tareas) y todos los commits con sus trailers `Plan-Id / Task-Id / Execution-Id` — trazabilidad completa desde cada línea de código hasta el run que la escribió. La URL del PR queda visible en la ficha del plan ( `pr_url1` ); si el PR no pudo abrirse, el motivo exacto queda en `pr_error` (p. ej. la rama base no existe en el remoto).

Revisa y mergea el PR en GitHub como con cualquier equipo humano. El ciclo queda cerrado: **idea** → **plan** → **agentes** → **PR** → **main**, con documentación en Confluence y Jira sincronizado por el camino.

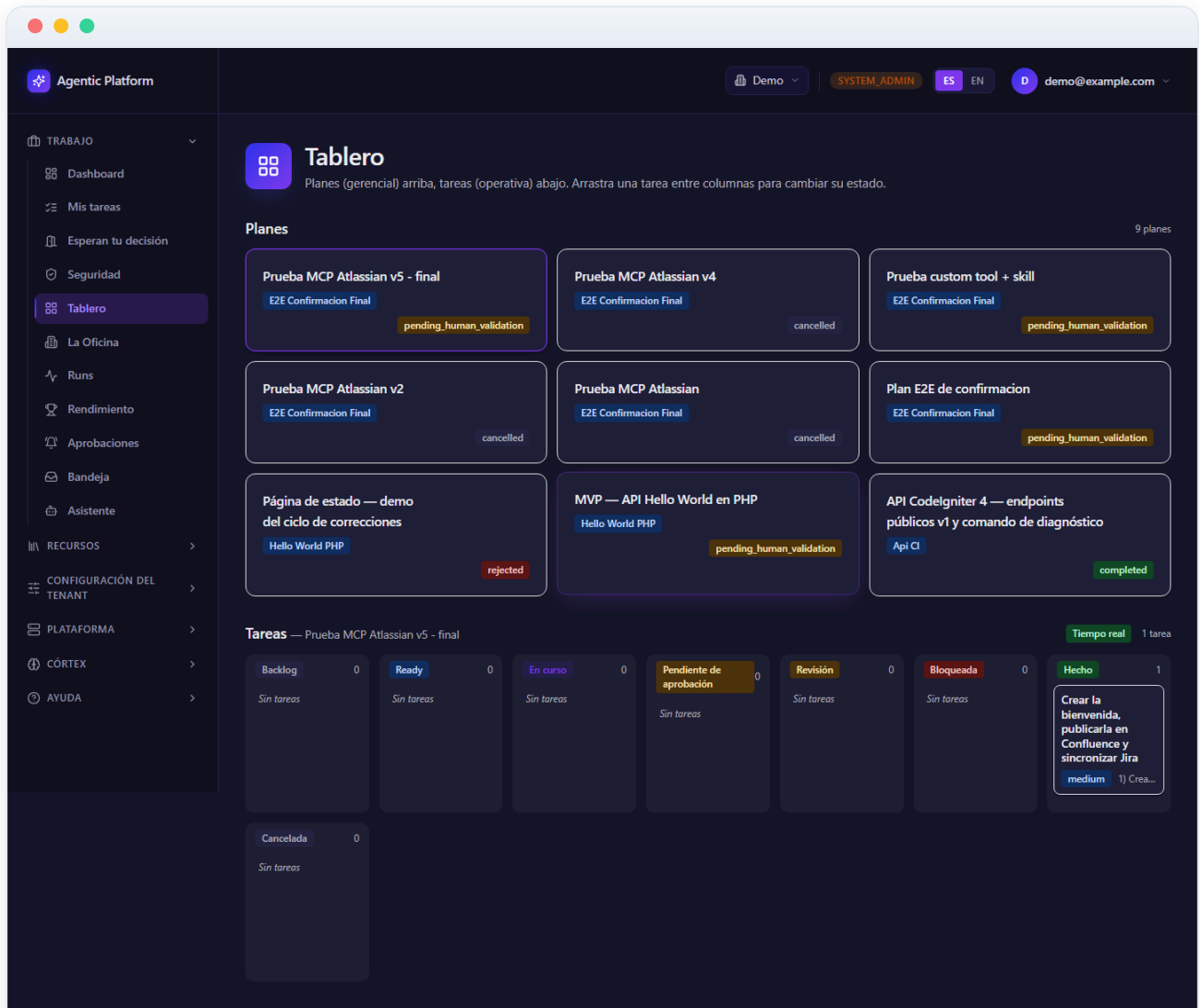


Figura 14.15 — El Pull Request en tu GitHub

## 16 Troubleshooting específico de CI4 + MCP

Los tropiezos más probables y su arreglo, por síntoma:

- **403 Filtered al conectar Context7** → falta `mcp.context7.com` en los dominios permitidos del proyecto (el egress es deny-by-default). El step `mcp_wire` del run lo dice literalmente.
- **mcp\_wire error contra mcp-atlassian** → el sidecar no está en la red `agentic-agents`, no arrancó, o las credenciales de Atlassian son inválidas (revisa `docker logs mcp-atlassian`).
- **unknown tool: atlassian....** → tools importadas pero no asignadas a ESE agente (Hub → Tools → Avanzadas).
- **composer install falla en el runtime** → comprueba que `composer` está en los comandos permitidos del proyecto y que el proyecto usa el runtime `php-phpunit` (que ya incluye las extensiones PHP necesarias, `intl` incluida).
- **El agente se queda en bucle leyendo ficheros** → tarea no autocontenida: reescribela para que cree sus insumos o garantíalos en el repo.
- **No llega el PR** → mira `pr_error` en la ficha del plan: casi siempre es la rama base inexistente en el remoto o el PAT sin permiso `repo`.
- **Nada se mueve** → **La Oficina** te dice en segundos si los agentes trabajan, esperan un humano (■ → resuélvelo en «Esperan tu decisión») o duermen (cola vacía: ¿el plan está aprobado y ejecutándose?).

Para profundizar: [docs/03-guides/recetas-mcp-tools-skills.md](#) (recetario completo de MCP/tools/skills), [docs/03-guides/configurar-mcp-server.md](#) (la pantalla de MCP a fondo) y el manual 13 (runs y supervisión).

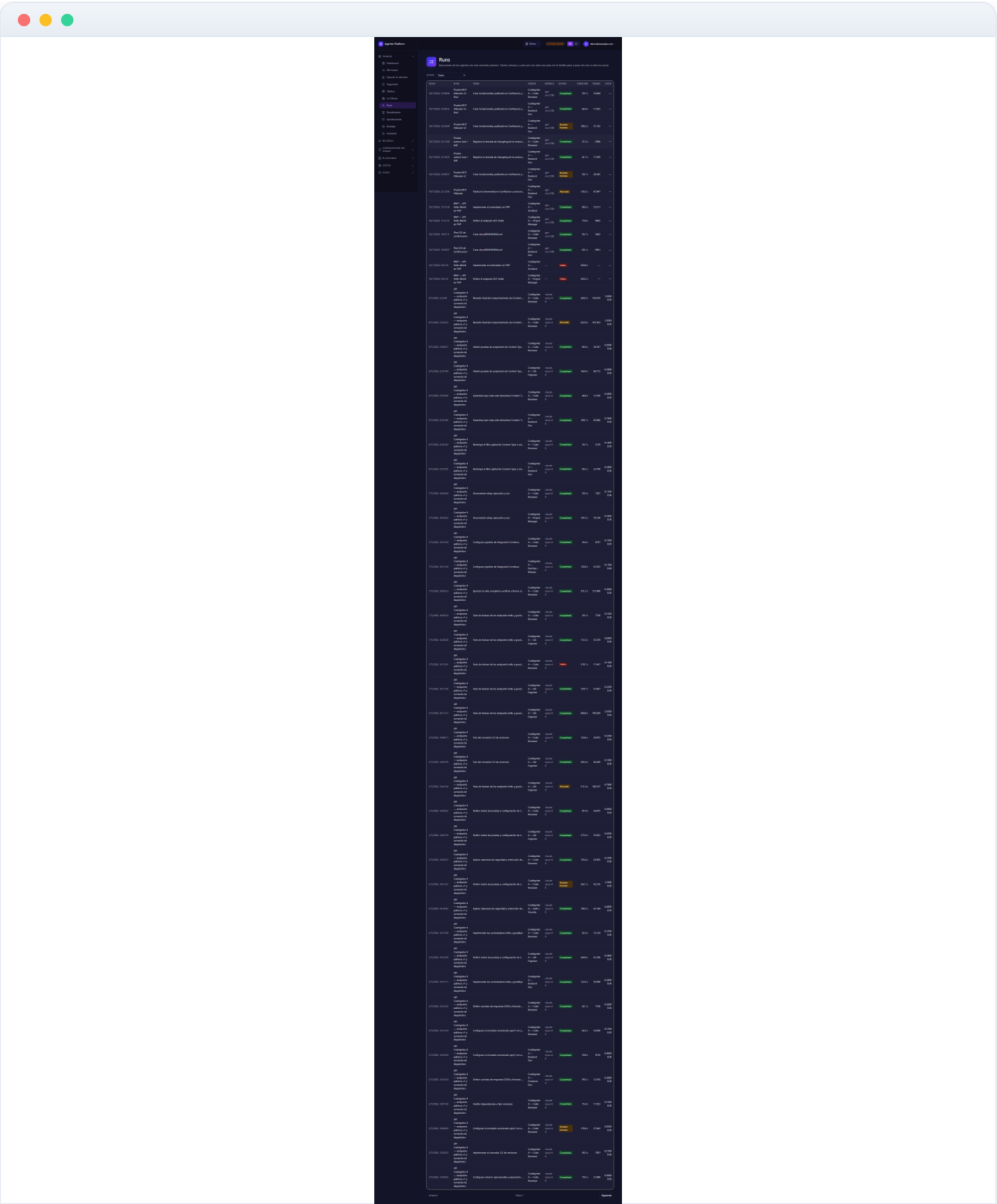


Figura 14.16 — Troubleshooting específico de CI4 + MCP